Artificial Intelligence of Things - Intelligent and Connected Systems

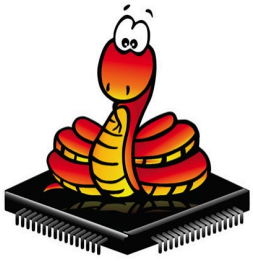
EECS E4764

Columbia University

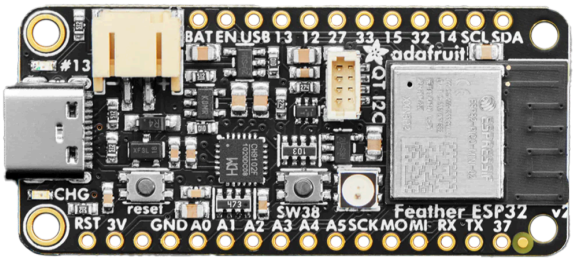
Prof. Xiaofan Jiang

Fall 2026

Lab 6 - Cloud, Zero-Shot and Few-Shot Human Activity Recognition with Foundation Models



MicroPython



Introduction

Human Activity Recognition (HAR) using wearable sensors is a core application in the field of Artificial Intelligence of Things (AIoT). Devices such as smartwatches, fitness trackers, and embedded IMUs in microcontrollers continuously generate sensor data such as acceleration and gyroscope signals. Accurately interpreting these signals makes it possible to monitor health, fitness, and daily activities in real time, powering applications from fall detection to personalized exercise feedback.

Traditionally, HAR systems rely on domain-specific machine learning models trained on large, labeled datasets. The emergence of Large Language Models (LLMs) and pre-trained foundation models (e.g., NormWear) introduce a new way to approach this task. By leveraging models trained on diverse, large-scale datasets, we can perform activity recognition in a zero-shot or few-shot manner. This greatly reduces the dependence on collecting and labeling large amounts of data, which is especially valuable in resource-constrained IoT settings.

Cloud computing also plays an important role in HAR. Since IoT devices often have limited computing power and storage, training and deploying models in the cloud is a common practice. In this lab, while we are not facing severe hardware limitations, we will still use the cloud to give you practical experience with training and serving HAR models remotely, similar to real-world AIoT deployments.

This lab will be slightly different from the previous lab as you will submit a mini report in addition to the checkpoint demos and code. The report will primarily consist of brief summaries and screenshots of your results, as demoing these things can be troublesome. What to include in the report will be described in each task.

Part 1: Zero-Shot Inference with LLM

☑ Task 1

Check the HARGPT [3] paper (an early and straightforward exploratory work on using LLM for HAR).

In this task, you will work with a standard benchmark dataset for HAR to become familiar with model inference. Start by downloading the UCI HAR dataset [1]. After downloading, locate and extract the raw IMU signals (accelerometer and gyroscope) found in the *Inertial Signals* folder. Next, implement a custom PyTorch Dataset and DataLoader to efficiently load the signals in the shape (batch_size, channels, seq_len) for both the training and test sets. Each signal file is stored as a .txt file with the shape [num_samples, 128]. You will need to stack the three accelerometer channels and three gyroscope channels to create a 6-dimensional input. Your Dataset should accept either “train” or “test” as an initialization argument to handle the corresponding split. This step ensures you are comfortable with working directly with raw sensor data and setting up data pipelines for model training and evaluation.

Here is some starter code showing what your dataset should look like (implement the TODOs yourself, you don’t have to exactly follow this template, just make sure that the output format is correct):

```python
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader
from pathlib import Path

HAR_LABELS = {1: "WALKING", 2: "WALKING_UPSTAIRS", 3: "WALKING_DOWNSTAIRS",
              4: "SITTING", 5: "STANDING", 6: "LAYING"}

CHANNEL_SETS = [
    "total_acc_x_{split}.txt",
    "total_acc_y_{split}.txt",
    "total_acc_z_{split}.txt",
    "body_gyro_x_{split}.txt",
    "body_gyro_y_{split}.txt",
    "body_gyro_z_{split}.txt",
]
```

```
class UCIHARSignals(Dataset):
    """
    Returns for __getitem__(i):
        x: torch.FloatTensor with shape [C, L]   (channels-first for Conv1d)
        y: int in [0..5]
        y_name: str (human-readable)
    """
    def __init__(self, root, split="train"):
        base = Path(root) / split / "Inertial Signals"
        # TODO: load and stack all channels -> [N, C, L]
        ...
        x = ...  # [N, C, L]
        # TODO (optional): normalize per channel; keep raw g for Tasks 2-5
        ...
        self.x = ...
        # TODO: load and remap labels to [0...5]
        ...
        self.y = ...

    def __len__(self):
        # TODO: return number of samples in dataset
        return ...

    def __getitem__(self, i):
        # TODO: return x tensor, y label, y label name
        x = ...                  # -> [C, 128]
        y = ...                  # -> scalar in [0..5]
        y_name = ...
        return x, y, y_name

if __name__ == "__main__":
    data_dir = "/path/to/UCI HAR Dataset"
    train_loader = DataLoader(UCIHARSignals(data_dir, "train"),
                              batch_size=32, shuffle=True)
    xb, yb, yname = next(iter(train_loader))
    print("x:", xb.shape)   # [B, C, L], e.g. [32, 6, 128]
    print("y:", yb.shape)   # [B]
    print("names:", yname[:3])
```

Task 2.1

In this task, you will learn how to interact with an LLM API for inference and prepare structured prompts for Human Activity Recognition (HAR). Set up the Gemini API to communicate with a cloud LLM. First, install the Python library for the Gemini API (python -m pip install google-genai) and obtain your API key from https://aistudio.google.com/apikey (the free tier has daily usage limits, which should be more than enough). Then test the following code to send user queries to the LLM and receive responses (similar to Lab 5):

```python
import asyncio
from google import genai

GEMINI_API_KEY = "Enter-API-KEY-HERE"

async def get_llm_response(prompt):
    full_response = ""
    async with genai.Client(api_key=GEMINI_API_KEY).aio as client:
        async for partial in await client.models.generate_content_stream(
            model="gemini-3.5-flash-lite",
            contents=prompt,
        ):
            if partial.text:
                full_response += partial.text
    return full_response

if __name__ == "__main__":
    prompt = input("Enter a prompt: ")
    print(asyncio.run(get_llm_response(prompt)))
```

After setting up the Gemini API, test it with various prompts to ensure it's working correctly.

After confirming your API connection, design a clear and structured prompt that encodes sensor data in natural language so the LLM can interpret it. Your prompt should include relevant metadata such as the device type (e.g., smartphone or ESP32), setting (e.g., lab or real world), and a list of candidate activities (e.g., walking, standing, sitting). Then, insert a small window of accelerometer and gyroscope data. A good template to follow is:

```python
"""
Device: Smartphone
Attached Location: {}
Candidate Activities: {walking, sitting, ...}
Data (Accelerometer X,Y,Z over time): [0.3, -0.1, ...]
Question: Which activity is being performed?
"""
```

Task 2.2

Chain-of-Thought (CoT) vs. Direct Answer. Compare two prompting styles for activity recognition with an LLM. For each chosen test sample, send two prompts: first, “Explain your reasoning step by step, then give the activity,” and second, “Give only the activity label.” Record both outputs for each case and assess whether the predicted labels are consistent across the two styles.

Checkpoint 1

Randomly get a sample from the test set of the UCI HAR Dataset loaded via the PyTorch Dataset, insert the data into a well-defined text prompt, and send a CoT and a direct answer prompt to LLM to get a classification of the activity.

Report: Repeat this for 10 sub-samples from the test set, compute accuracy for each prompting style, and briefly summarize whether there is any difference in performance or stability between step-by-step reasoning and direct answers.

Task 3

Connect your ESP32 and capture a few seconds of accelerometer data matching the windows of the UCI HAR inputs (same window: 128 samples at 50 Hz, in g; 3 channels only). Use the same prompt structure you developed earlier to query the LLM with this ESP32 data and display the predicted activity on the OLED. Add this as an additional command (e.g., classify_HAR) to your lab 5 interactive interface, using the Gemini API from Task 2.1.

Checkpoint 2

Collect a sample with ESP32, use the same prompt structure you developed earlier to query the LLM with this ESP32 data and display the predicted activity on the OLED

Report: Collect 10 samples from your ESP32 (at least 1 for each of the 6 classes) and query the LLM with the same prompt structure. Compare these results with the UCI-based prompts to evaluate whether the model's behavior generalizes to real hardware signals collected in your environment. Keep the window length consistent with the UCI dataset to make the comparison fair.

Part 2: Zero-Shot & Few-Shot Inference with Pre-Trained Foundation Model on the Cloud

Make sure you are familiar with Google Cloud Compute Engine, as the following tasks are expected to run on a Compute Engine VM instance.

Task 4

We will use Flask to create a web server to allow ESP to interact with the cloud. If you haven't worked with Flask before, here is a simple guide.
https://pythonbasics.org/flask-rest-api/

Run the flask server and test with your browser to see if you can connect to it. To reach it from outside the VM, start Flask with host="0.0.0.0" and add a VPC firewall rule that allows TCP traffic on its port (e.g., 5000), then open http://<VM external IP>:5000. Notice that if you close the SSH application, your program will automatically shut down instead of running in the background. You can use a tool called "screen" to keep the program running after the terminal is closed.
https://linuxize.com/post/how-to-use-linux-screen/

Create one RESTful interface on your Flask server to receive accelerometer data from ESP, and save it as persistent files (we recommend jsonl as it is readable and easy to append to). Modify your ESP program so that it will send accelerometer data to the cloud server's IP instead of your local interactive interface. You can use *urequests* library to interface with the Flask endpoint.

Checkpoint 3
Collect a sample with ESP32, send it to the cloud server, and show that it is stored properly.

Task 5.1

In this task, you will work with an open-source pre-trained foundation model for Human Activity Recognition to understand how zero-shot inference works in practice. Start by cloning the **NormWear** GitHub repository from this link and follow the instructions provided to set up the environment (install PyTorch with pip; the pytorch conda channel used in config_env.sh is discontinued). The pre-trained checkpoints are on the repository's Releases page (v1.0.0-alpha). If you want to speed up training and inference, you may launch a virtual machine on Google Cloud with a standard GPU (such as an NVIDIA L4). However, be mindful of cloud costs, always stop the VM when not in use to avoid unnecessary charges. If you cannot provision a GPU-enabled instance, NormWear runs entirely on CPU as well (give the VM at least 16 GB of RAM, e.g., e2-standard-4). Once your environment is ready, load the pre-trained model as described in the repository's documentation and run inference on the full UCI HAR test set (note: recent SciPy versions removed scipy.signal.cwt, so use the model's PyTorch wavelet option instead). For zero-shot classification, the model expects (1) description of the task of interest, (2) the answer options, (3) the signal data. This will give you hands-on experience using a pre-trained foundation model for activity recognition without any model training. As a reference, you can expect a baseline zero-shot accuracy of around 16% on the UCI HAR dataset. Running the model on the CPU typically takes about 10–20 minutes per evaluation cycle.

Report: State the input(s) you fed to NormWear. Report the accuracy you got for zero-shot classification you got on the UCI HAR test set, and the time it took to run (CPU or GPU).

Task 5.2

In this task, you will adapt the pre-trained **NormWear** model to the UCI HAR dataset by adding a simple trainable classification head on top of its frozen encoder. The goal is to get hands-on experience with a lightweight fine-tuning strategy, which is often enough to significantly boost accuracy without training a large model from scratch.

First, use the NormWear encoder to extract signal embeddings for around 5–10 samples per class in the UCI HAR training set. Use the get_embedding function of NormWearModel (signal_encode is the zero-shot path). It takes a batch of raw sensor windows (shape [B, C, L], e.g., 3 channels × 128 time steps; pass sampling_rate=50) and returns [B, C, P, 768]; average over patches and channels to get one 768-dimensional embedding per window. Run this for both the training and test sets, and store the resulting embeddings along with their labels.

Next, define a simple PyTorch model with just one fully connected (linear) layer. The input dimension of this layer should match the embedding size (for example, if NormWear outputs a 768-dimensional embedding, your input shape should be in_features=768), and the output dimension should match the number of activity classes in UCI HAR (6 classes). You do not need to modify or train the encoder, only the classification head will be trained.

Train this linear layer using the extracted embeddings as inputs and the activity labels as targets. Use a standard cross-entropy loss function, and train for about 100 epochs in a few-shot setting (e.g., the 5–10 samples per class you encoded before). This step is computationally lightweight since you're only optimizing a small layer, and it can be done entirely on CPU.

Finally, evaluate the model on the test set embeddings and report the classification accuracy. Even with this minimal fine-tuning, you should see a significant improvement, often around a 40% boost in accuracy compared to the zero-shot baseline. This demonstrates how a simple classification head can effectively adapt a pre-trained foundation model to a specific target dataset.

Report: Your accuracy on the test set before and after.

Task 5.3

In this task, you will build a complete end-to-end inference pipeline that replaces the previous LLM-based activity classification with a more efficient NormWear encoder plus your fine-tuned linear probe. This approach brings the system closer to a deployable IoT inference pipeline by combining real sensor data with a lightweight edge-to-cloud processing flow.

Start by collecting accelerometer data from your ESP32 device (the ADXL345 has no gyroscope) in the same format and window length as used in Task 3 (e.g., 3 channels × 128 time steps). Send this to your cloud server endpoint. Once the server receives the data, load the saved NormWear encoder (frozen) and your trained linear classification head from the previous task. Convert the incoming sensor window into a tensor with shape [1, C, L] and pass it through the NormWear encoder to obtain a fixed-length embedding vector (e.g., [1, 768]). Then feed this embedding into your fine-tuned linear layer to generate activity class logits and predicted labels. The server should return the predicted activity (e.g., “walking” or “standing”) to the ESP32 and display it on the OLED.

Checkpoint 4

Collect a sample with ESP32, send it to the cloud server, get embedding with NormWear, run inference with your PyTorch model, return the prediction to ESP32 and display on OLED.

Lab 6 Checkpoints:

1. Randomly get a sample from the test set of the UCI HAR Dataset loaded via the PyTorch Dataset, insert the data into a well-defined text prompt, and send a CoT and a direct answer prompt to LLM to get a classification of the activity.
2. Collect a sample with ESP32, use the same prompt structure you developed earlier to query the LLM with this ESP32 data and display the predicted activity on the OLED
3. Collect a sample with ESP32, send it to the cloud server, and show that it is stored properly.
4. Collect a sample with ESP32, send it to the cloud server, get embedding with NormWear, run inference with your PyTorch model, return the prediction to ESP32 and display on OLED.

Submission:

You are required to submit your code and a brief lab report.

Please organize your code in a structured folder and submit a .zip file. At minimum, it should contain:

- Task 1 – PyTorch Dataset and DataLoader implementation for the UCI HAR dataset.
- Task 2 – Python scripts for:
    - LLM API prompt generation and inference (both CoT and direct answer).
    - Prompt formatting and test sample evaluation.
- Task 3 – ESP32 code modifications to collect accelerometer data and display the LLM classification result on the OLED.
- Task 4 – Flask server implementation for receiving and storing accelerometer data from ESP32.
- Task 5 –
    - Script for running zero-shot inference with NormWear.
    - Script for extracting embeddings and training a linear classification head.
    - End-to-end inference pipeline integrating ESP32 data, NormWear, and the linear probe.

Your report should be a PDF containing:

- **Task 2 (LLM prompts)**
  - A few example CoT and direct answer outputs for the test samples.
  - Accuracy comparison and short reflection on differences between the two prompting strategies.
- **Task 3 (ESP32 + LLM)**
  - Summary table of 10 ESP32 samples tested.
  - Observations on generalization from UCI HAR to real signals.
- **Task 5.1 (Zero-Shot NormWear)**
  - Input/format used for inference.
  - Accuracy and runtime (CPU or GPU).
- **Task 5.2 (Few-Shot Fine-Tuning)**
  - Accuracy before and after training the linear classification head.
- **Task 5.3 (End-to-End Pipeline)**
  - A short description of how your pipeline works.
  - Example input → prediction results (e.g., screenshots or logs).

Include relevant screenshots, figures, or tables where appropriate.
Keep your explanations concise, bullet points are fine for most sections.

References

[1] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN 2013. Dataset: UCI HAR

[2] Y. Luo, Y. Chen, A. Salekin, and T. Rahman, “Toward Foundation Model for Multivariate Wearable Sensing of Physiological Signals,” ACM Transactions on Computing for Healthcare, 2026. arXiv:2412.09758

[3] S. Ji, X. Zheng, and C. Wu, “HARGPT: Are LLMs Zero-Shot Human Activity Recognizers?” 2024. arXiv:2403.02727